```
public class NetworkClient
{
    private readonly EmailServerSettings _settings;
    public NetworkClient(EmailServerSettings settings)
    {
        _settings = settings;
    }
}
```

This example might feel a little contrived, but it's common to find this sort of chain of dependencies. In fact, if you *don't* have it in your code, it's probably a sign that your classes are too big and aren't following the SRP.

So how does this affect the code in `RegisterUser`? The following listing shows how you now have to send an email if you stick to `new`ing up objects in the handler.

Listing 8.3 Sending email without DI when you create dependencies manually

```
string RegisterUser(string username)
{
    var emailSender = new EmailSender(      ❶
        new MessageFactory(),               ❷
        new NetworkClient(                  ❸
            new EmailServerSettings         ❹
            (                               ❹
                Host: "smtp.server.com",    ❹
                Port: 25                    ❹
            ))                              ❹
        );
    emailSender.SendEmail(username);        ❺
    return $"Email sent to {username}!";
}
```

❶ To create EmailSender, you must create all its dependencies.

❷ You need a new MessageFactory.

❸ The NetworkClient also has dependencies.

❹ You're already two layers deep, but there could feasibly be more.